WEEK 6 · DAY 2 · RULE BOOK

When Will It Break? Remaining Useful Life + Quiz 4

Follow it step by step. Every common error and its fix is here.

Your goal today: make the remaining-useful-life calls for a battery, an amplifier and a fuel tank; find and fix the bug that sends the fuel back to day 0; then check the battery call against what really happened. **13 green ✅ ticks** to collect. Time: about 45 minutes, then Quiz 4. Keep this page open next to Colab.

Before you start (2 minutes)

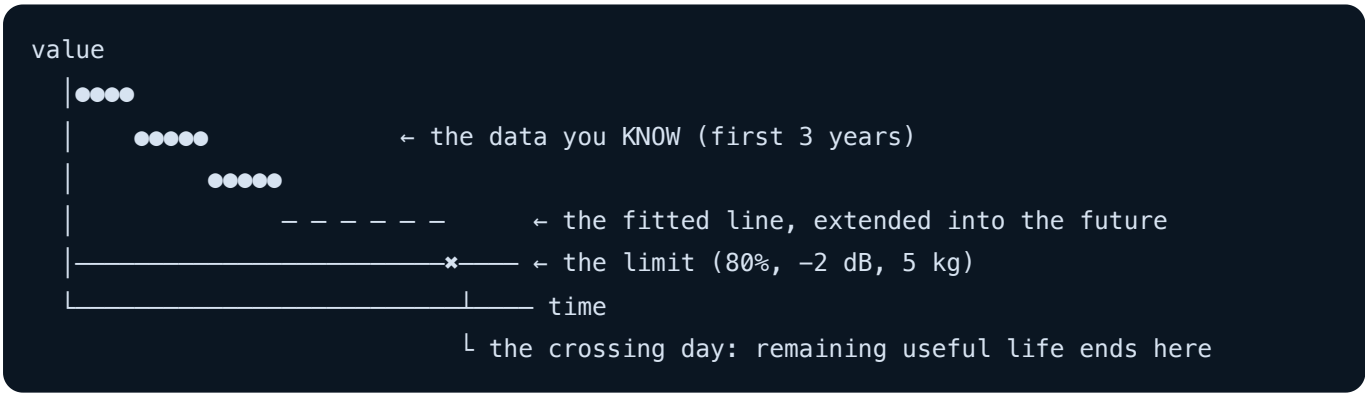
☐ Open `W6D2-exercise` in Colab → **File** → **Save a copy in Drive**.

☐ Run **STEP 0**. Wait for ✅ `Ready!`

☐ Run the three ▶️ cells under STEP 0 (data + limits, the plot, today's tools). Everything below needs them.

☐ Run every cell in order. Part B needs your A2 table.

The idea in one picture



Remaining useful life (RUL) = the time from today until the trend crosses the limit. The danfo driver's version: *(what's left above the limit) ÷ (how fast it's going)*.

The three parts and their limits

Part	Column	Limit	Why that limit
🔋 Battery	`battery_capacity_pct`	**80 %**	below it the battery can't safely carry the eclipse-season discharge

Part	Column	Limit	Why that limit
📡 Amplifier	`twta_gain_db`	**−2 dB**	below it the link budget loses its margin
⛽ Propellant	`propellant_kg`	**5 kg**	the reserve kept for the end-of-mission de-orbit burn

The limits are **known up front** (`degradation_thresholds.csv`). We forecast *when* they are reached.

The exercise, step by step

Task	What you do	The tool you need
A1	Battery slope per year	`DAYS_PER_YEAR = 365`
A2	The three calls from 3 years of data	`COLUMN = "threshold"` (a column name, in quotes)
A3	Why the original printed 1 Jan 2021 for propellant	`True` or `False`
A4	Solve the propellant line for 5 kg	`(5.0 − intercept) / slope`
A5	The fuel-gauge sum	`RESERVE_KG = 5.0`
B1	When did the battery really cross?	`FLOOR = 80.0`
B2	Score the 3-year call two ways	`KEY = "30_day_mean"`
B3	The call from 4 starting points	`YEARS = [1, 2, 3, 4]`
B4	Is the fade speeding up or slowing down?	`"slows"` or `"speeds up"`
C1	Share of the fade on eclipse days	`ECLIPSE = 1`
C2	How far past the data the amplifier call reaches	`LAST = −1`
C3	The honest report	`"A"`, `"B"` or `"C"`
🌶️ **Bonus**	Bend the line: a curve instead	`DEGREE = 2`

The code you'll use

```
known = degr.iloc[:365 * 3]                          # ONLY the first 3 years
model, (slope, intercept) = fit_trend(known["battery_capacity_pct"])
slope * 365                                          # % per year

day, line = day_of_threshold(known["battery_capacity_pct"], 80.0)
degr.index[0] + pd.Timedelta(days=day)               # day number → date
# day is None means: no crossing inside the search window

(5.0 - intercept) / slope                            # a straight line, solved for the
limit
crossing_dates(bat, 80.0, window=30)                 # first dip · 30-day mean · last
good day
```

When things go wrong: error → fix

You see	It means	Fix
`AssertionError: ✍️ Fill in … first`	A `___` is still there	Replace it, run the cell again
`AssertionError: ✍️ COLUMN is the NAME of a column`	You typed `80` or `threshold` without quotes	Type `"threshold"` (text, in quotes)
`KeyError: '30 day mean'` (or similar)	The key is spelt differently	Copy it exactly from the B1 print: `"30_day_mean"`
`AssertionError: ✍️ Finish A2 first / B1 first`	You skipped a task	Do that task, then come back
`NameError: name 'fit_trend' is not defined`	You skipped the ▶️ tools cell	Run the three ▶️ cells under STEP 0
`TypeError: unsupported operand … 'NoneType'`	`day_of_threshold` returned `None` (no crossing) and you used it as a number	That's the honest "no crossing" answer: solve the line instead (A4)
`TypeError: 'int' object is not iterable` in B3	`YEARS = 4`	`YEARS` is a **list**: `[1, 2, 3, 4]`
A date one day off from your neighbour's	One of you used the formula (rounds down), the other `day_of_threshold` (first whole day below)	Same line, same answer: a day either way doesn't matter for a 7-year call

You see	It means	Fix
Everything is weird	–	**Runtime → Restart session and run all**

Rules of thumb you'll keep forever

1. **Fit on the past only.** Make the call with the data you'd have had on that day.
2. **"No crossing" is an answer.** Never let a function turn "never" into day 0. Check for `None`.
3. **Sanity-check with a simple sum.** Left above the limit ÷ rate per year. If it disagrees with the model, one of them has a bug.
4. **Define "crossed" before you score.** First dip, 30-day mean or last good day: write down which, and why.
5. **Backtest the method** from several starting points. Note which way it errs (early or late).
6. **Report a window, never a single day**, and say how far past the data you reached.
7. **Act before the crossing**, and re-estimate every time new data arrives.

🏁 **Finished?** Run `scoreboard()`. Then try the 🌶️ curve, and get ready for **Quiz 4** (phones down, no AI, 20 minutes).